



Full length article

Impact assessment of third-party library vulnerabilities through vulnerability reachability analysis

[Zhizhuang Jia](#) , [Chao Yang](#)  , [Pengbin Feng](#) , [Xiaoyun Zhao](#) , [Xinghua Li](#) , [Jianfeng Ma](#) 

School of Cyber Engineering, Xidian University, Xi'an, 710119, Shaanxi, China

Received 10 March 2025, Revised 24 April 2025, Accepted 23 May 2025, Available online 9 July 2025, Version of Record 15 July 2025.

 [What do these dates mean?](#)



Show less 

 Share  Cite

<https://doi.org/10.1016/j.cose.2025.104546> 

[Get rights and content](#) 

Abstract

Modern software development increasingly relies on third-party libraries (TPLs) to accelerate development, yet this practice introduces security risks when vulnerabilities emerge in dependencies. When a new TPL vulnerability is disclosed, project maintainers must assess whether their projects are affected, a process that demands considerable developer effort. Vulnerability reachability analysis automates this process by evaluating whether a client program's control flow can reach the vulnerable TPL function. Despite the numerous advantages, however, existing vulnerability reachability analysis fail to account for the impact of path constraints, leading to false alarms by incorrectly identifying unreachable vulnerable functions as reachable. In this paper, we propose CPVRA, a novel impact assessment framework designed to reduce false alarms by incorporating a satisfiability assessment of path constraints into the analysis process. We evaluated CPVRA using a benchmark dataset comprising 201 test cases derived from 6 widely used TPLs and 24 dependent client programs. The results show that CPVRA reduced false alarms by 40.9% compared to Dep-scan and by 39.5% compared to standard vulnerability reachability analysis.

Additionally, CPVRA demonstrated high computational efficiency, with an average analysis time of 15.3 s per client program.

Introduction

In recent years, the software supply chain-based development model has gained widespread popularity among developers. It is estimated that 97% of software projects utilize the functionality provided by third-party libraries (TPLs) (Synopsys, 2022) to accelerate development. However, alongside the efficiency benefits of not having to “reinvent the wheel”, TPLs also introduce numerous security concerns. For instance, vulnerabilities within the Apache log4j2 library impacted over 35,000 downstream projects (Google, 2021), and the Open Worldwide Application Security Project (OWASP) lists the use of vulnerable components among its top 10 security risks (OWASP, 2021).

While updating the TPLs on which software relies can effectively reduce the risk of attacks, in many cases, however, developers cannot simply update a dependency when a new fix is released, as doing so may introduce conflicts within the software ecosystem (Bogart et al., 2016). Bug fixes, code refactoring, or feature additions can break the library’s application program interface(API) (Kim et al., 2011), preventing updating them in a timely manner. A study (Kula et al., 2018) of 4600 software projects and 2500 TPLs revealed that 81.5% of projects rely on outdated dependencies, these outdated TPLs pose a significant security risk to software projects.

The presence of a vulnerable dependency in a project does not inherently imply that the project is compromised. Zapata et al. (2018) manually analyzed 60 npm projects flagged for security risks and found that over 70% of reported vulnerabilities were false alarms. Updating TPLs that do not affect project security not only wastes developers’ valuable time but also delays the mitigation of genuine vulnerabilities, thereby elevating the risk of security breaches. As a result, developers frequently evaluate whether a vulnerability impacts their project before proceeding with dependency updates. However, this process can be particularly time-consuming in large projects (Cobleigh et al., 2018). Thus, automating the risk assessment of TPL vulnerabilities has become an increasingly urgent challenge.

Existing Techniques. In recent years, multiple impact assessment methods have been developed to support developers in automatically evaluating risks associated with TPL vulnerabilities. These approaches are broadly categorized into three groups: metadata-based, fuzzing-based, and static analysis-based techniques. Metadata-based methods query TPLs in vulnerability databases, such as the National Vulnerability Database (NVD) (NVD, 2024), and evaluate severity based on metadata, such as the Common Vulnerability Scoring System (CVSS) score (Group, 2024). Despite their computational efficiency, metadata-based methods suffer from a high rate of false alarms. Fuzzing-based methods evaluate the risk of TPL vulnerabilities by generating inputs that trigger TPL vulnerabilities (Zhou et al., 2024, Iannone et al., 2021, Kang et al., 2022, Chen et al., 2024). The primary advantage of this approach is it has the highest precision. However, since fuzzing cannot exhaustively explore an application’s entire input space, it can only be used to confirm the impact of an identified vulnerability. Static analysis-based methods employ techniques like vulnerability reachability analysis(VRA) (Research, 2023, dep-scan, 2025, Zhang et al., 2024, Ponta et al., 2020,

Nielsen et al., 2021, Borzacchiello et al., 2022, Mir et al., 2023), taint analysis (semgrep, 2023), and symbolic execution (Ma et al., 2020) to examine the code of TPLs and client programs, providing a more accurate risk assessment result. Nevertheless, depending on the static analysis techniques employed, these methods may encounter scalability issues when applied to large and complex projects.

Among existing methods, vulnerability reachability analysis has emerged as a prominent approach for assessing the impact of TPL vulnerabilities. It evaluates the impact of vulnerabilities by determining whether the control flow of the client program can reach the vulnerable function in a TPL. This straightforward yet effective approach offers higher analysis accuracy than metadata-based solutions and greater execution efficiency than other static analysis-based methods. Additionally, it can complement the results of fuzzing-based methods.

Limitations of Existing Vulnerability Reachability Analysis. Despite the numerous advantages, however, existing vulnerability reachability analyses fail to consider the impact of path constraints on reachability, which compromises the accuracy of the result. In a comprehensive investigation of software projects within the Maven ecosystem, Wu et al. (2023) discovered that up to 82.4% of paths from the client program to vulnerable functions in TPLs featured path constraints. Among these constraints, 39.5% were of reference type, posing challenges for constraint solvers. If the path constraints on a call path cannot be satisfied, the call will not occur. However, existing solutions typically generate function call graphs through static analysis frameworks such as Soot (Vallée-Rai et al., 2010) and WALA (wala, 2005), which do not consider path constraints in the construction process. This limitation results in the inclusion of non-existent paths in the generated function call graphs, which can ultimately lead to false alarms. The prevalence of false alarms not only squanders developers' efforts and prolongs the resolution time for real vulnerabilities but also undermines developers' trust in impact assessment techniques, as some may opt to disregard all warnings.

Research Problem Identifying non-existent paths requires tracking condition variables and evaluating the satisfiability of path constraints based on their values. One intuitive way to solve this problem is by using symbolic execution. However, as call paths expand, the number of condition variables requiring tracking increases significantly, and determining satisfiability becomes increasingly challenging, especially when these variables are subject to complex constraints. Ma et al. (2020) mitigates this problem by manually extracting path constraints from bug reports, which is difficult to automate as it not only requires access to TPL vulnerability patch reports but also deep expertise in the TPL and the underlying causes of the vulnerabilities. DroidReach (Borzacchiello et al., 2022) also attempts to improve the precision of vulnerability reachability analysis through symbolic execution, however, it only uses symbolic execution to resolve indirect call targets, disabling path constraint solving to reduce the performance overhead. In addition to symbolic execution, Steady (Research, 2023) enhances the accuracy of vulnerability reachability analysis by recording the execution traces of the TPL. Nevertheless, its primary use is limited to resolving indirect call targets, as exploring the entire input space of an application remains challenging. **As such, improving the accuracy of vulnerability reachability analysis while maintaining computational efficiency remains an open research problem.**

Our idea. Rather than attempting to prove the satisfiability of all branching constraints, we focus on identifying unsatisfiable constraints along the call path. This approach reduces the number of

condition variables that need to be tracked. Additionally, we track the concrete values of condition variables through constant propagation (Wegman and Zadeck, 1991), rather than their symbolic states, simplifying the determination of satisfiability.

Our approach. Since we cannot predict which branch constraints on the calling path are unsatisfiable, we cannot determine which condition variables to trace. Nevertheless, we can still increase the probability of finding an unsatisfiable path constraint by tracking the state of the widely accessed condition variable. We observe that applications utilizing TPLs typically require multiple API function calls to achieve specific functionalities. Most TPLs, except for very simple ones, need to maintain state information — such as configuration parameters and callback function pointers — across these API calls. To facilitate this, a context object is passed to API functions, either as an explicit parameter (e.g., the File pointer in `fwrite`) or an implicit parameter (the “this” pointer in C++). This context object stores information about the current state of the TPL, and its value can significantly influence the execution path of the API functions. By inferring the value of the context object, we can identify call paths that fail to satisfy the required constraints, thereby enhancing the accuracy of the result. In this process, we face two key challenges: context object identification and context object value inference.

Each TPL may define multiple context objects, which can change across different versions. Manual labeling of these objects is both time-consuming and error-prone. To address this, we propose an automated context object identification method. Our approach leverages the observation that context objects are passed as arguments during API calls from client programs. Consequently, we initially classify all memory objects referenced by these arguments as candidate context objects. However, this preliminary classification frequently includes extraneous objects (e.g., input/output buffers), introducing noise that degrades analytical efficiency. To mitigate this, we implement type fuzzy matching and heuristic rules to systematically exclude non-context objects, ensuring robust and precise identification.

Sophisticated analysis methods enable precise inference of context object values but incur significant computational overhead. Simpler methods, while efficient, sacrifice accuracy in value resolution. To balance precision and efficiency, we introduce context object-guided constant propagation (COGCP). Our framework first derives function summaries for each TPL function, detailing their interactions with context objects. The propagation engine then performs interprocedural analysis exclusively on functions that access context objects while conservatively tracking side effects in other functions. This approach ensures analytical soundness without compromising scalability.

We developed a prototype called CPVRA. To evaluate its effectiveness, we constructed a test set of 201 items based on 6 widely used TPLs and 24 client programs dependent on them. Each item represents a client program and a corresponding TPL vulnerability to be assessed. Results demonstrate that CPVRA reduced false alarms by 40.9% and 39.5% compared to `dep-scan` and standard vulnerability reachability analysis, respectively. Moreover, the average time taken to complete the analysis for a client program was 15.3 s, highlighting its high computational efficiency.

In short, we make the following contributions:

1. We propose a novel impact assessment framework that enhances the accuracy of vulnerability reachability analysis through path constraint satisfiability evaluation.
2. We developed a prototype called CPVRA and subjected it to a test set of 201 test items. The results demonstrate that CPVRA can reduce false alarms by 40.9% compared to dep-scan and by 39.5% compared to standard vulnerability reachability analysis. Additionally, CPVRA demonstrated high computational efficiency, with an average analysis time of 15.3 s per client program.
3. We made our work open source to facilitate other researchers. The source code of the prototype is at <https://github.com/NullRefExceptions/SCA-Prun> ↗.

Access through your organization

Check access to the full text by signing in through your organization.

Access through **your organization**

Section snippets

Related work

In recent years, various impact assessment techniques have emerged to automate the evaluation of risks posed by vulnerabilities in TPLs. These methods generally fall into three categories: metadata-based, static analysis-based, and fuzzing-based approaches.

Metadata-based impact assessments are widely used in software composition analysis tools such as OWASP Dependency-Check (OWASP, 2023) and GitHub Dependabot (GitHub, 2023). These methods query TPLs in vulnerability databases like NVD (NVD, 2024 ...

Motivation

In this section, we present an illustrative example to demonstrate the impact of path constraints on vulnerability reachability.

Listing 1 illustrates a classic scenario of a client program using the TPL. The main function of the client program first calls the TPL_Init function to initialize the TPL, where the context object is initialized with the mode field set to MODE_R. This context object is then passed when the client program calls TPL_FuncA. Within TPL_FuncA, the execution path diverges ...

Design

In this section, we present the design details of CPVRA. Fig. 2 illustrates the overall architecture of the framework, which is divided into three key phases: pre-processing, function summary generation, and vulnerability reachability analysis.

In the pre-processing phase, the LLVM intermediate representation (IR) (Lattner and Adve, 2004) of the inspected application and TPLs are extracted and linked to generate *Module_{Linked}*, which serves as input for subsequent phases. During the function ...

Evaluation

We conduct an experimental evaluation to assess the effectiveness of our method in analyzing the impact of TPL vulnerabilities. This evaluation seeks to answer the following research questions:

- RQ1: How does the accuracy and efficiency of the CPVRA compare to other methods for assessing the impact of TPL vulnerabilities? ...
- RQ2: How effective is context object identification in reducing non-context objects and enhancing analysis efficiency? ...
- RQ3: How does context object-guided constant propagation ...

...

Discussion

In this section, we discuss the limitations of CPVRA and propose directions for future improvements.

CPVRA assesses vulnerability impact by determining the reachability between the client program and the vulnerable function. The reliability of its results depends on the accuracy of the generated function call graphs. Despite our best efforts, these graphs cannot be guaranteed to be entirely accurate. One reason for this is potential errors in the initial function call graph. We use TypeDive (Lu ...

Conclusion

We propose a novel impact assessment scheme that integrates a satisfiability assessment of path constraints into the standard vulnerability reachability analysis. The scheme first employs static analysis to identify context objects within third-party libraries, then validates the runtime values of their internal fields through constant propagation. Subsequently, it determines the satisfiability of the path constraints based on the analysis results and identifies false alarms when the path ...

CRedit authorship contribution statement

Zhizhuang Jia: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal

analysis, Data curation, Conceptualization. **Chao Yang:** Writing – review & editing, Funding acquisition, Conceptualization. **Pengbin Feng:** Writing – review & editing, Conceptualization. **Xiaoyun Zhao:** Writing – review & editing, Conceptualization. **Xinghua Li:** Writing – review & editing, Conceptualization. ...

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. ...

Acknowledgment

The authors would like to thank the editors and anonymous reviewers for their constructive feedback and insightful suggestions that helped to significantly improve the quality of this work. This work is supported by the Key Program of Cyberspace Administration of Shaanxi Province - Information Security Research Institute Program of Shaanxi, Technology Innovation Leading Program of Shaanxi, China (Grant No. C023340004, Program No. 2024QCY-KXJ-171), Research and Development Program of CETC 36th ...

[Recommended articles](#)

References (40)

A.A. Ahmad *et al.*
Trimmer: an automated system for configuration-based software debloating
IEEE Trans. Softw. Eng. (2021)

L.O. Andersen
Program analysis and specialization for the C programming language
(1994)

Bogart, C., Kästner, C., Herbsleb, J., Thung, F., 2016. How to break an API: cost negotiation and community values in...

L. Borzacchiello *et al.*
Reach me if you can: on native vulnerability reachability in android apps

Chen, Z., Hu, X., Xia, X., Gao, Y., Xu, T., Lo, D., Yang, X., 2024. Exploiting Library Vulnerability via Migration...

A. Cobleigh *et al.*
Identifying, prioritizing and evaluating vulnerabilities in third party code

dep-scan
OWASP dep-scan

(2025)

GitHub

Dependabot

(2023)

Google

Understanding the impact of Apache log4j vulnerability

(2021)

C.S.I. Group

Common vulnerability scoring system

(2024)

A. Hazimeh *et al.*

Magma: A ground-truth fuzzing benchmark

Proc. ACM Meas. Anal. Comput. Syst. (2020)

Hind, M., 2001. Pointer analysis: Haven't we solved this problem yet?. In: Proceedings of the 2001 ACM SIGPLAN-SIGSOFT...

E. Iannone *et al.*

Toward automated exploit generation for known vulnerabilities in open-source libraries

Kang, H.J., Nguyen, T.G., Le, B., Păsăreanu, C.S., Lo, D., 2022. Test mimicry to assess the exploitability of library...

Kim, M., Cai, D., Kim, S., 2011. An empirical investigation into the role of API-level refactorings during software...

R.G. Kula *et al.*

Do developers update their library dependencies? An empirical study on the impact of security advisories on library migration

Empir. Softw. Eng. (2018)

C. Lattner

LLVM 3.0 type system rewrite

(2011)

C. Lattner *et al.*

LLVM: A compilation framework for lifelong program analysis & transformation

Lu, K., Hu, H., 2019. Where does it go? refining indirect-call targets with multi-layer type analysis. In: Proceedings...

Ma, W., Chen, L., Zhang, X., Feng, Y., Xu, Z., Chen, Z., Zhou, Y., Xu, B., 2020. Impact analysis of cross-project bugs...

There are more references available in the full text version of this article.

Cited by (2)

[Network-Centric Security Awareness in Software Development: Integrating Vulnerability Patterns, Developer Practices, and Organizational Controls ↗](#)

2026, International Journal of Safety and Security Engineering

[Fine-grained Detection of Java Cross-library Vulnerability Propagation by Extracting Semantic Constraints from Security Patches ↗](#)

2026, IEEE Transactions on Dependable and Secure Computing

[View full text](#)

© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

